



Processos

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Conceito de processo
- Escalonamento de processos
- Operações sobre processos
- Comunicação entre processos
- Comunicação em sistemas cliente-servidor

Conceito de processo

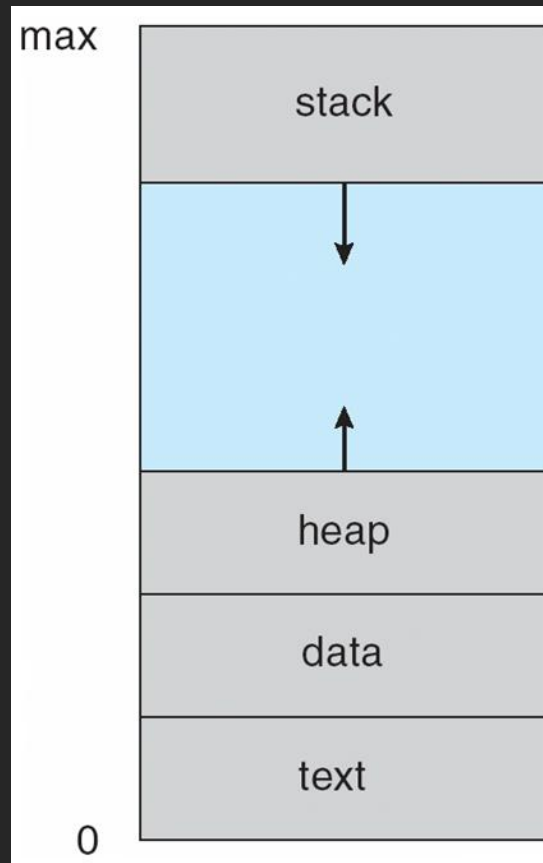
- SO coordena a execução de uma variedade de programas
 - Job: nome dado a esses programas em um sistema batch
 - Programa/tarefa: nome dado em um sistema de tempo compartilhado
 - Silberschatz et. al: job = processo
- Processo: programa em execução
 - Programa: entidade passiva
 - Processo: entidade ativa
 - Execução do processo deve progredir de forma sequencial

Conceito de processo

- Um processo possui:
 - Código/seção de texto
 - Programa (arquivo executável) também possui
 - Contador de programa (e demais registradores): atividade corrente
 - Pilha: dados temporários
 - Parâmetros, endereços de retorno, variáveis locais
 - Seção de dados: variáveis globais
 - Heap: memória alocada dinamicamente

Conceito de processo

- Estrutura de um processo na memória:

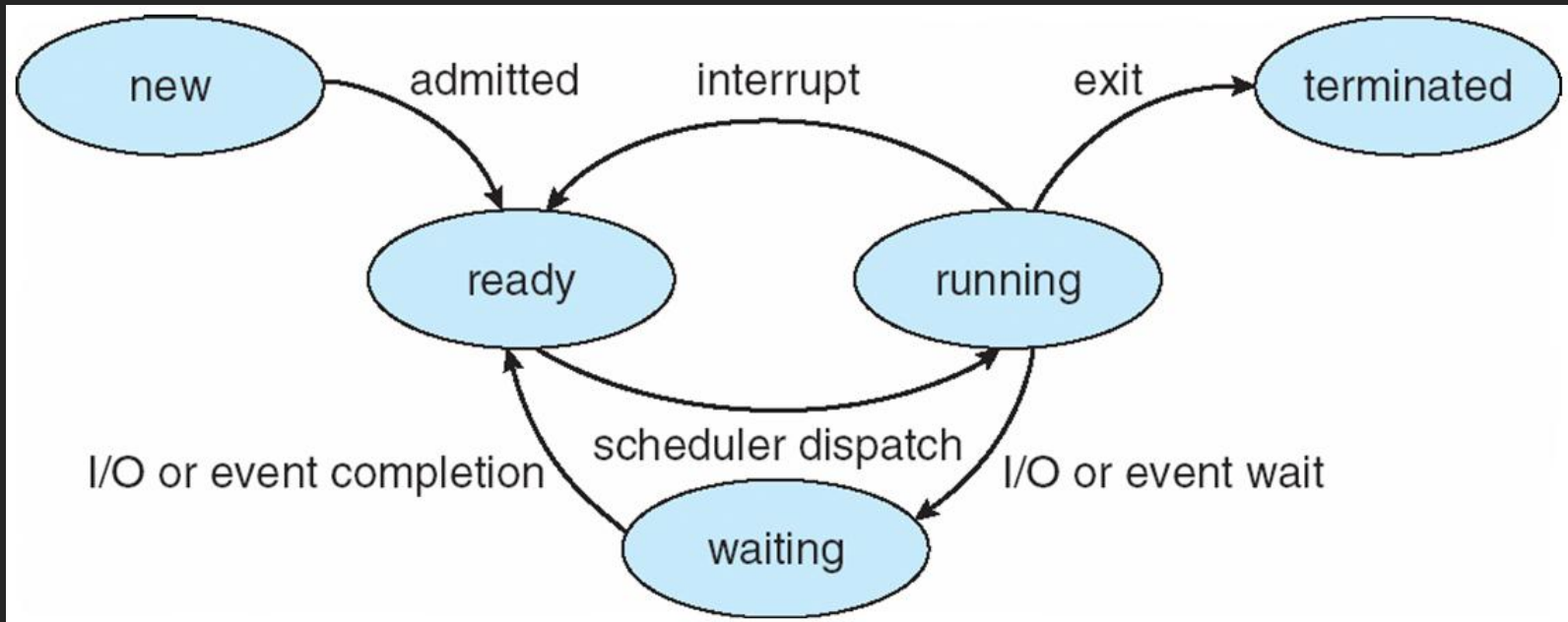


Conceito de processo

- O estado de um processo está relacionado à atividade corrente do processo
 - Durante execução, processo possui seu estado alterado
- Possíveis estados:
 - Novo (new): o processo está sendo criado
 - Executando (running): instruções sendo executadas na CPU
 - Esperando (waiting): aguardando por E/S ou evento
 - Pronto (ready): aguardando ser atribuído a um processador
 - Terminado (terminated): o processo finalizou a execução

Conceito de processo

- Diagrama de estados:



Conceito de processo

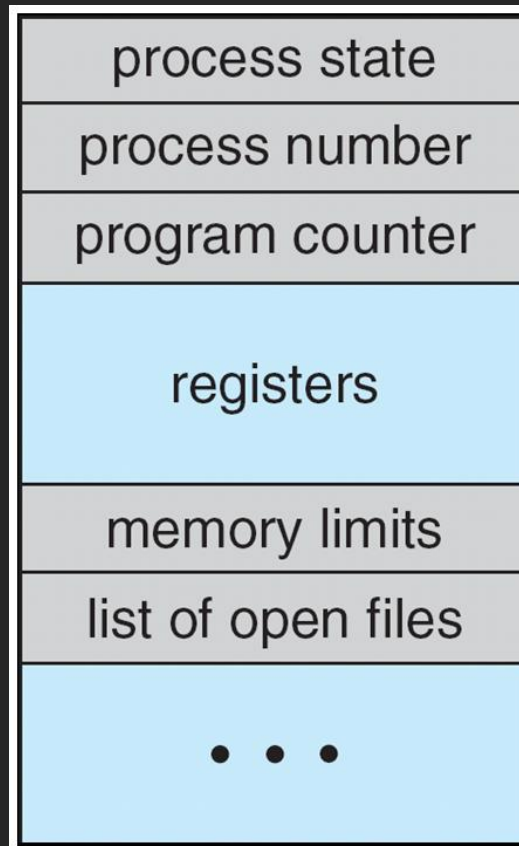
- O bloco de controle de processo (BCP/PCB) mantém informação associada a cada processo
 - Estado: estado atual do processo
 - Contador de programa: endereço da próxima instrução
 - Registradores da CPU: necessários para continuar execução após interrupção
 - Informações de escalonamento de CPU: prioridade, ponteiro para fila de escalonamento
 - Informação de gerenciamento de memória: registradores base, limite, tabela de páginas, tabela de segmentos

Conceito de processo

- (cont.)
 - Informação de contabilidade: tempo de CPU, tempo total, tempo bloqueado, etc.
 - Informação de status de E/S: lista de dispositivos alocados ao processo, lista de arquivos abertos, etc.
- Processos multithread: várias linhas de execução separadas
 - BCP também deve suportar informações de cada thread

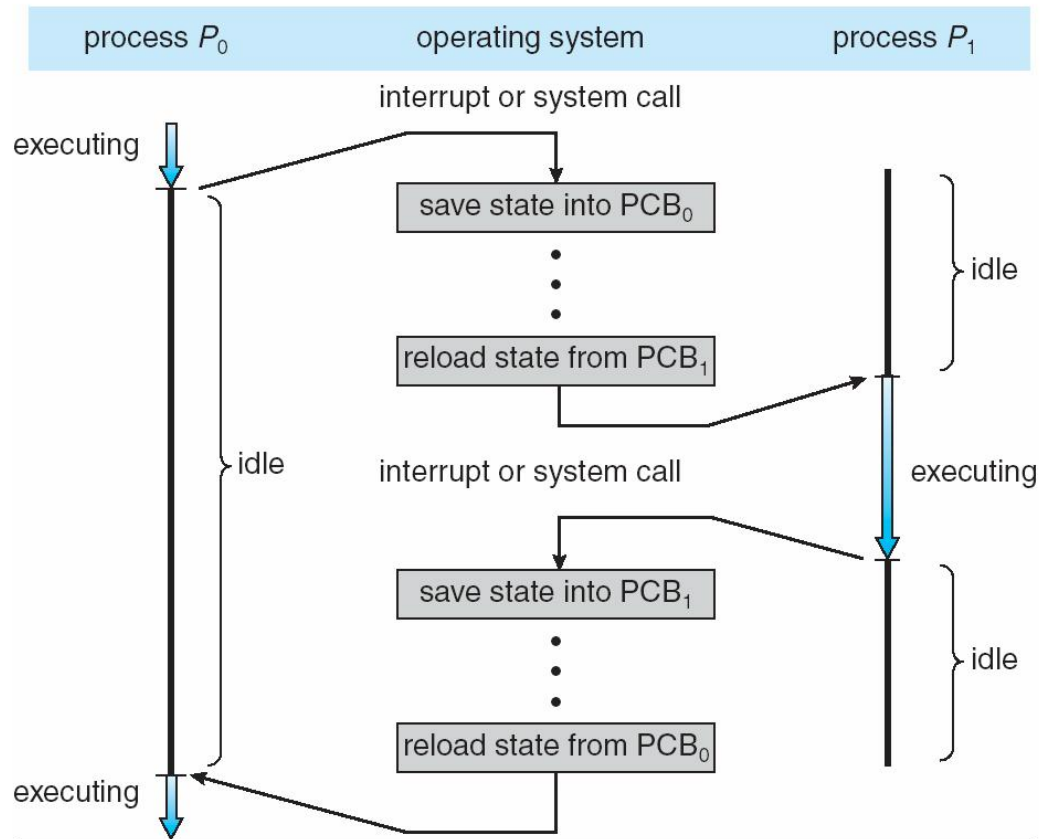
Conceito de processo

- BCP:



Conceito de processo

- Uso dos registradores da CPU contidos no BCP:



Conceito de processo

- BCP no Linux: struct task_struct
 - include/linux/sched.h
 - Alguns campos:

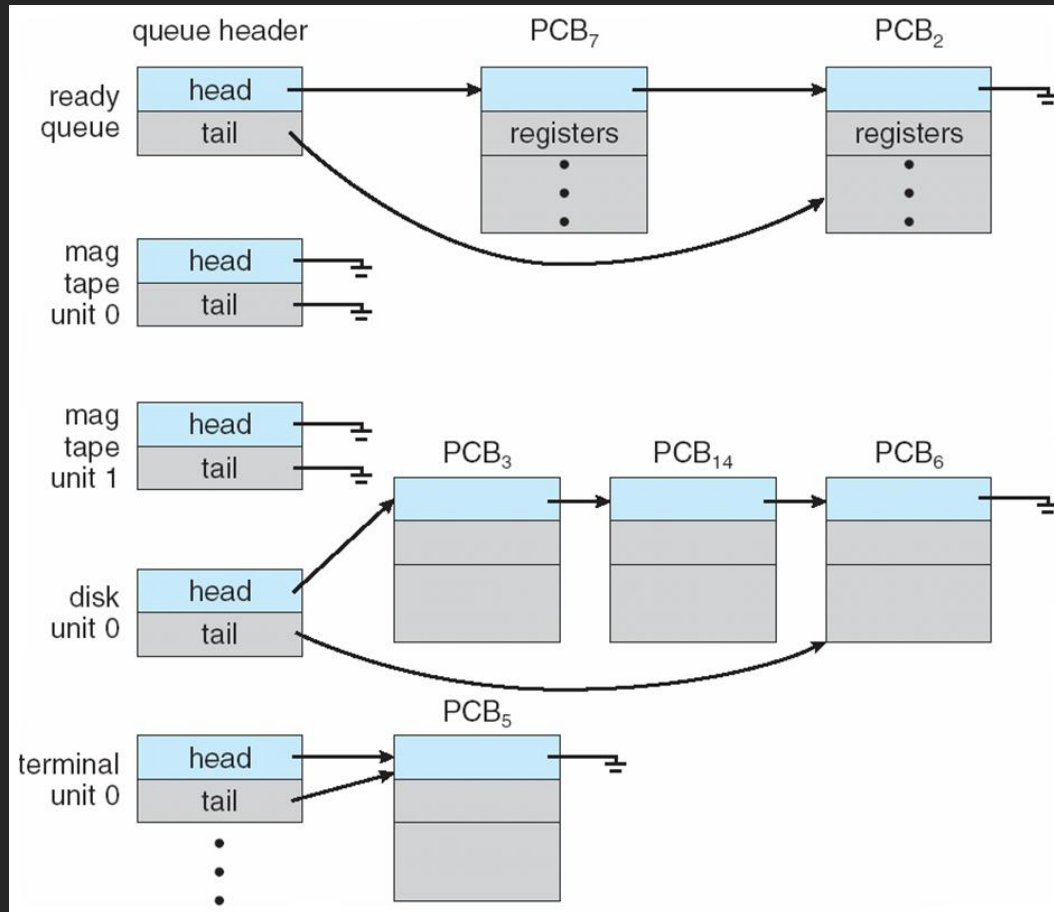
```
pid_t pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

Escalonamento de processos

- Objetivo da multiprogramação: maximizar uso da CPU
 - Vários programas em memória
 - Fila de prontos: processos disponíveis para executar na CPU
 - Escalonador faz escolha
- Há diversas filas de escalonamento:
 - Fila de tarefas/jobs: envolve todos os processos no sistema
 - Fila de prontos: processos em memória
 - Aguardando para ser executado na CPU
 - Diversas outras filas: dispositivos de E/S
- Durante execução, processo migra de uma fila para outra

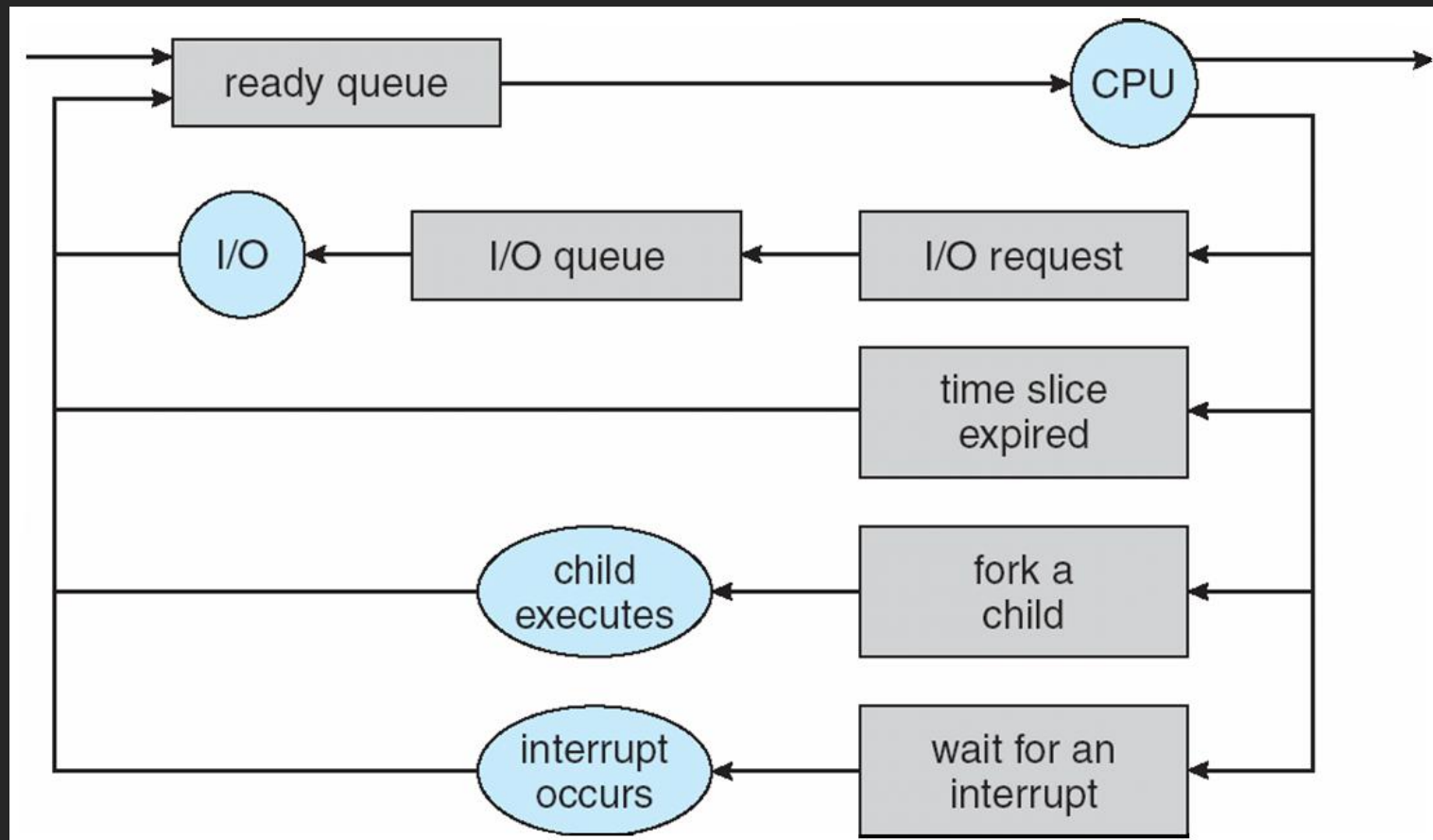
Escalonamento de processos

- Filas de prontos e de dispositivos de E/S:



Escalonamento de processos

- Diagrama de enfileiramento:



Escalonamento de processos

- Escalonador de curto prazo (de CPU): seleciona entre processos prontos para executar na CPU
 - Executado com bastante frequência (várias vezes por segundo): deve ser bastante rápido
 - Em geral: pelo menos 1 vez a cada 100 ms
 - Se escalonador gasta 10 ms para decidir: $10/(100+10) = 9\%$ da CPU é “perdida” apenas para escalonar

Escalonamento de processos

- Escalonador de longo prazo (escalonador de jobs):
seleciona processos a serem carregados na memória para execução
 - Executado com pouca frequência (até minutos): quando memória fica disponível
 - Controla grau de multiprogramação (número de processos na memória)
 - Grau estável: número de processos que são criados = número de processos terminados

Escalonamento de processos

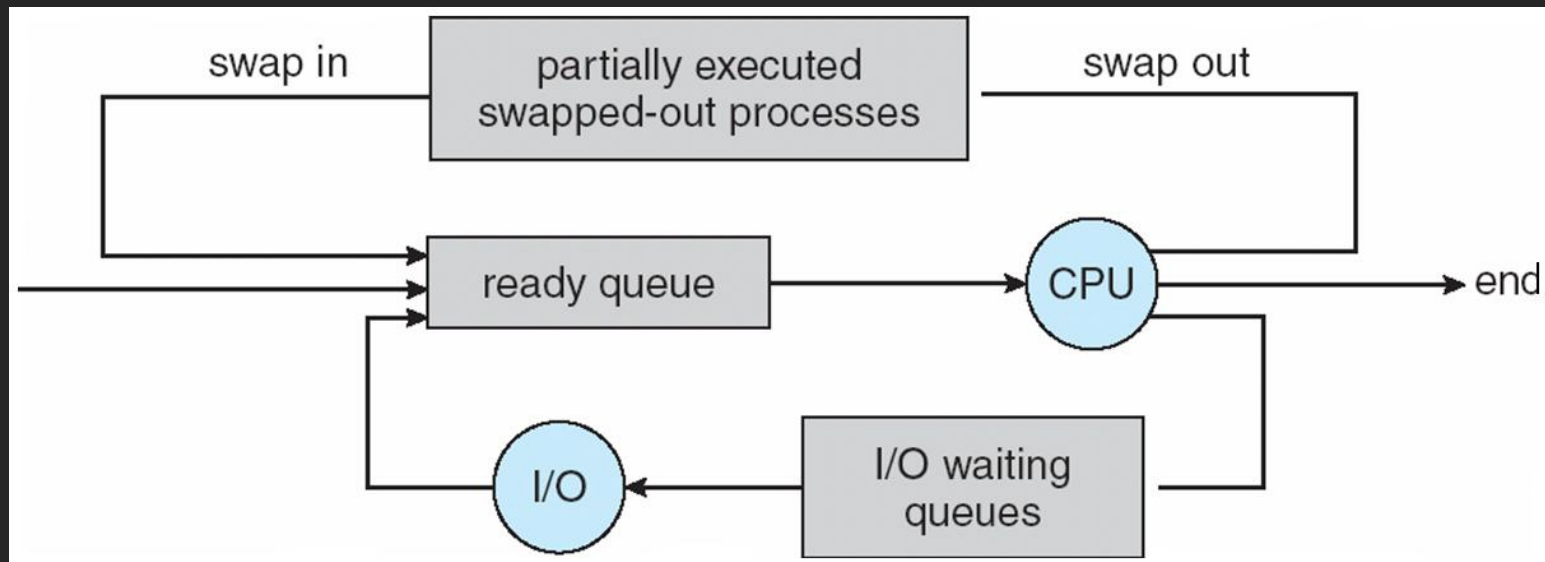
- Conceitos:
 - Processo limitado por E/S (I/O bound): gasta mais tempo realizando E/S do que executando na CPU
 - Processo limitado por CPU (CPU-bound): gasta mais tempo executando na CPU do que realizando E/S
- Escalonador de longo prazo deve selecionar processos de maneira balanceada (I/O-bound + CPU-bound)
 - Não sobrecarregar uma fila e esvaziar outra: ineficiência no uso dos recursos
 - Ausente em alguns sistemas (UNIX, Windows)
 - Usam apenas escalonador de curto prazo

Escalonamento de processos

- SOs de tempo compartilhado empregam escalonador de médio prazo
 - Swapping: processos podem ser removidos da memória (swap out) e reintroduzido (swap in)
 - Também controla o grau de multiprogramação

Escalonamento de processos

- Inclusão do escalonador de médio prazo no diagrama de enfileiramento:



Escalonamento de processos

- Contexto de um processo: valores dos registradores, estado do processo, informações de gerenciamento de memória, etc.
 - Armazenado no BCP
- Quando há troca de processo na CPU:
 - SO deve salvar estado da CPU para o processo que a está deixando
 - SO deve restaurar o estado da CPU para o processo que a está ganhando
 - Troca de contexto: salvar + restaurar estados para viabilizar troca de processo

Escalonamento de processos

- Tempo gasto na troca de contexto: overhead extra
 - Sistema não faz nenhum trabalho “útil”
 - Tempo depende de suporte de hardware
 - Ex.: Sun UltraSPARC possui vários conjuntos de registradores
 - Troca de contexto: mudar ponteiro para conjunto ativo
 - Mas se há mais processos que conjuntos de registradores...

Operações sobre processos

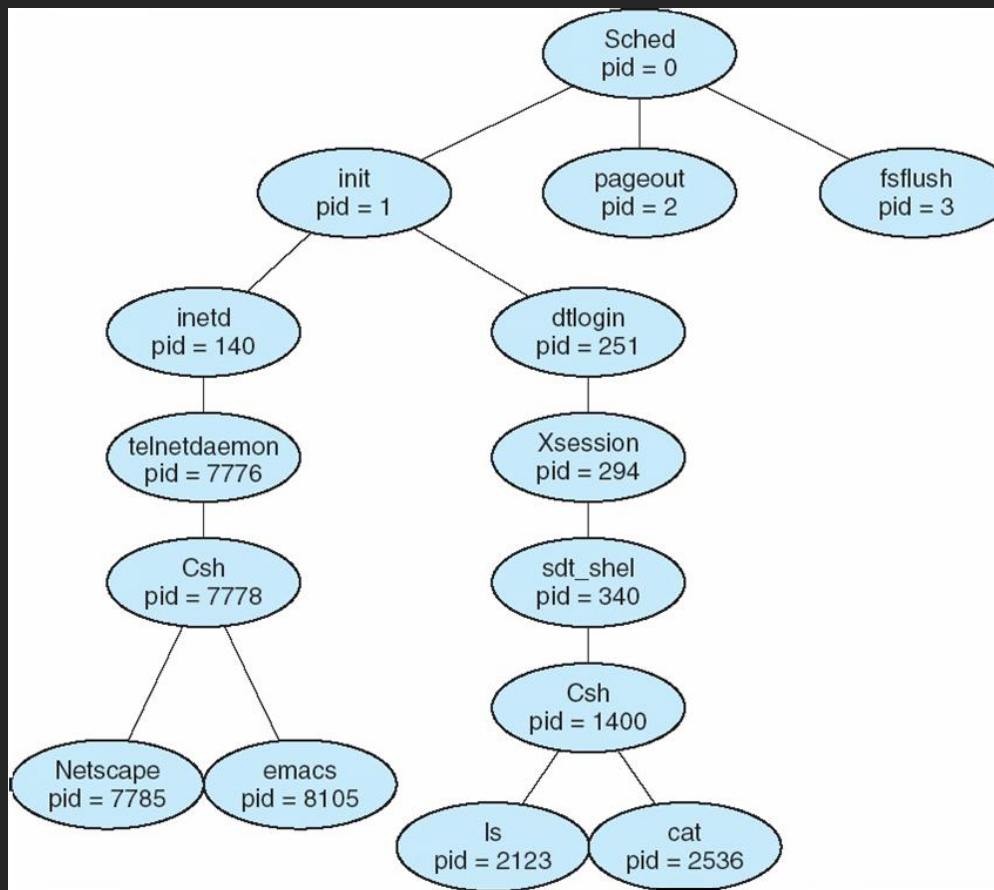
- Criação de processos:
 - Processo pai cria processos filhos, que criam outros processos: árvore de processos (ex: pstree)
 - Processos são identificados e gerenciados através de um identificador de processo: pid
 - Compartilhamento de recursos entre pai e filhos: tudo, subconjunto de recursos ou nada
 - Pai pode inicializar argumentos do filho

Operações sobre processos

- (cont.)
 - Execução: pai e filhos executam concorrentemente ou pai espera filhos terminarem
 - Espaço de endereçamento:
 - Filho é duplicado do pai (ex.: fork)
 - Torna comunicação mais fácil entre pai e filho
 - Filho possui outro programa carregado (ex.: exec)

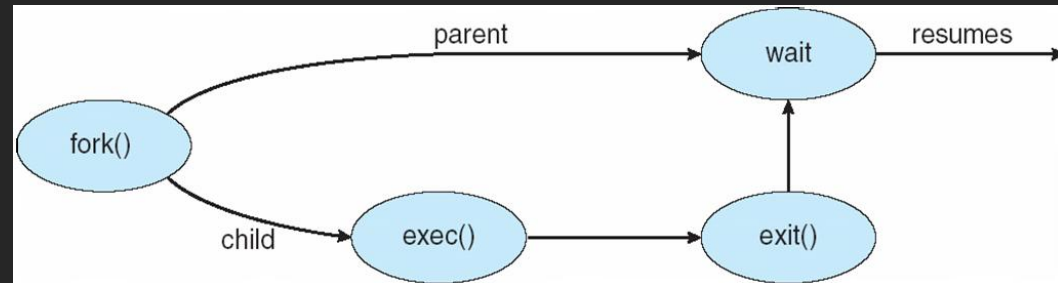
Operações sobre processos

- Árvore de processos no Solaris:



Operações sobre processos

```
int main()
{
    pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```



Operações sobre processos

```
#include < stdio.h >
#include < windows.h >
int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    // allocate memory
    ZeroMemory(&si, sizeof(si));
    Si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    // create child process
    if (!CreateProcess(NULL, "C: \\ WINDOWS \\ system32 \\ mspaint.exe", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    // parent will wait for the child to complete
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");
    // close handles
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Operações sobre processos

- Término de processos:
 - Processo solicita término ao SO término (ex: exit)
 - Filho pode retornar status para o pai via chamada wait
 - Recursos do processo são desalocados pelo SO
 - Processo pai pode terminar execução do filho (abort):
 - Filho excedeu recursos alocados
 - Tarefa do filho não é mais necessária
 - Pai está terminando

Operações sobre processos

- (cont.):
 - Alguns SOs (ex.: VMS) não permitem filho continuar quando pai terminal: término em cascata
 - No UNIX:
 - Processo pede para terminar via `exit()`
 - Pai aguardando: `wait()` retorna pid do filho que terminou
 - Pai termina e filhos podem continuar executando
 - Novo pai: processo `init`

Comunicação entre processos

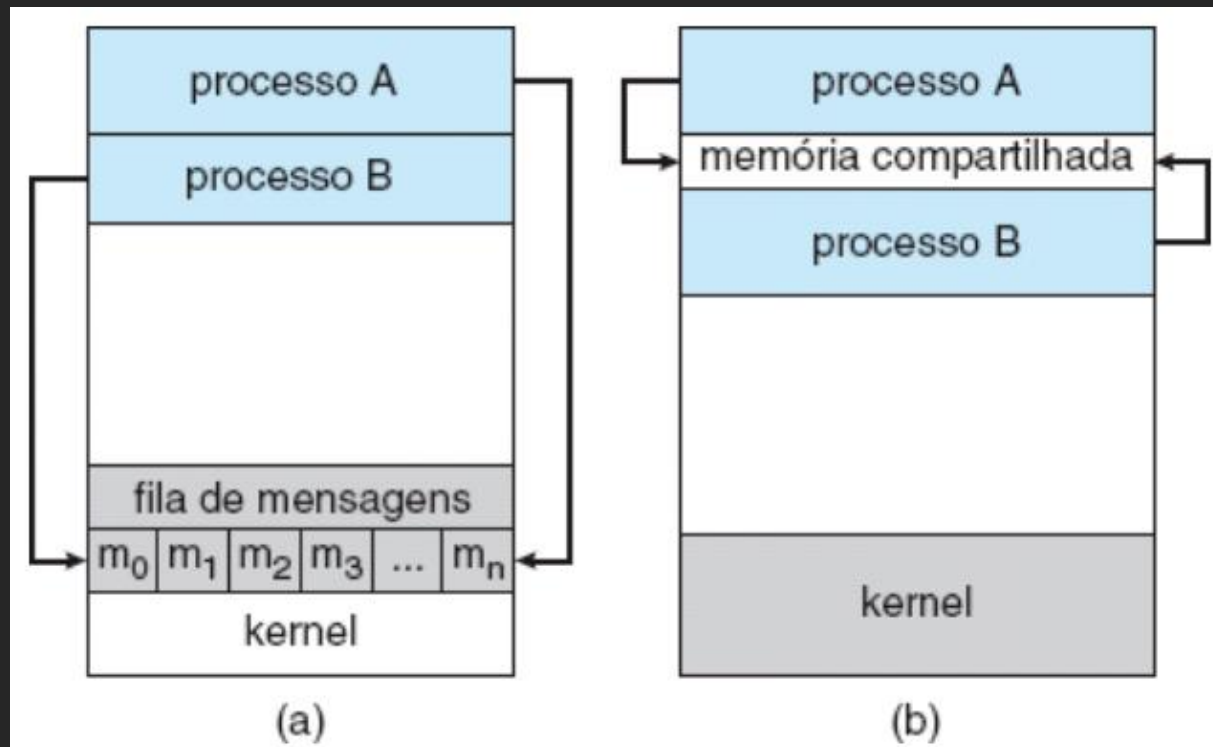
- Um processo pode ser independente ou cooperativo
 - Processo cooperativo: pode afetar a (ou ser afetado pela) execução de outro processo
- Razões para uso de processos cooperativos:
 - Compartilhamento de informação
 - Acelerar computação
 - Modularidade
 - Conveniência

Comunicação entre processos

- Processos cooperativos necessitam de comunicação: IPC (interprocess communication)
 - Dois modelos de IPC:
 - Memória compartilhada: comunicação ocorre através de acesso à região de memória compartilhada entre os processos
 - Passagem de mensagens: comunicação ocorre através do envio e recebimento de mensagens entre os processos

Comunicação entre processos

- Modelos de IPC por passagem de mensagens (a) e memória compartilhada (b):



Comunicação entre processos

- Problema do produtor-consumidor: processo produtor produz a informação que é consumida por um processo consumidor
 - Usa um buffer preenchido pelo produtor e esvaziado pelo consumidor
 - Buffer ilimitado: inexiste na prática
 - Buffer limitado: possui tamanho fixo pré-estabelecido

Comunicação entre processos

- Solução do produtor-consumidor usando memória compartilhada:

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Buffer vazio: $in == out$
- Buffer cheio: $((in + 1) \% BUFFER_SIZE) == out$
- Buffer pode armazenar, no máximo, $BUFFER_SIZE - 1$ itens

Comunicação entre processos

- Produtor:

item nextProduced;

while (true) {

 /* produce an item in nextProduced */

 while (((in + 1) % BUFFER SIZE) == out)

 ; /* do nothing */

 buffer[in] = nextProduced;

 in = (in + 1) % BUFFER SIZE ;

}

Comunicação entre processos

- Consumidor:

item nextConsumed;

```
while (true) {  
    while (in == out)  
        ; // do nothing  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER SIZE ;  
    /* consume the item in nextConsumed */  
}
```

Comunicação entre processos

- Passagem de mensagens: mecanismo para processos se comunicarem e sincronizarem suas ações
 - Comunicação não recorre à variáveis compartilhadas
 - IPC provê duas operações:
 - Envia/send(msg): tamanho de msg é fixo ou variável
 - Recebe/receive(msg)
 - Quando P e Q desejam comunicar, eles necessitam:
 - Estabelecer link/canal de comunicação entre eles
 - Trocar mensagens via send/receive

Comunicação entre processos

- Implementação do link de comunicação
 - Física: memória compartilhada, barramento, rede de computadores
 - Lógica:
 - Comunicação direta ou indireta?
 - Comunicação síncrona ou assíncrona?
 - Bufferização automática ou explícita?

Comunicação entre processos

- Nomeação de processos: meio de um processo se referir a outro
 - Duas formas: comunicação direta ou indireta
 - Comunicação direta: emissor e receptor devem ser identificados
 - Simétrica: especifica emissor e receptor
 - `send(P, message)`: envia mensagem ao processo P
 - `receive(Q, message)`: recebe mensagem do processo Q
 - Assimétrica: especifica emissor, mas não receptor
 - `receive(id, message)`: id receberá a identificação do processo que enviou a mensagem

Comunicação entre processos

- (cont. nomeação):
 - Propriedades do link na comunicação direta:
 - Links são estabelecidos automaticamente
 - Um link é associado com exatamente um par de processos
 - Entre cada par de processos há exatamente um link
 - O link pode ser unidirecional, mas é usualmente bi-direcional
 - Dificuldades com a comunicação direta:
 - Processos devem conhecer as identidades dos demais
 - Alterar a identidade de um processo: problema

Comunicação entre processos

- (cont. nomeação):
 - Comunicação indireta: mensagens são enviadas para caixas postais (mailbox) ou portas
 - Caixa postal: abstrai objeto onde processos podem colocar mensagens e de onde também podem retirar mensagens
 - Cada caixa postal tem identificação única: no POSIX, um inteiro
 - Primitivas:
 - `send(A, message)`: envia mensagem para caixa postal A
 - `receive(A, message)`: recebe mensagem da caixa postal A

Comunicação entre processos

- (cont. nomeação):
 - Propriedades do link na comunicação indireta:
 - Um link é estabelecido entre dois processos somente se ambos têm caixa postal compartilhada
 - Um link pode ser associado a mais de dois processos
 - Entre cada par de processos pode haver um número diferente de links, cada link correspondente a uma caixa postal

Comunicação entre processos

- (cont. nomeação):
 - Caixa postal pode ser propriedade de processo ou do sistema
 - Operações com caixas postais:
 - Criação
 - Envio e recebimento de mensagens através dela
 - Término

Comunicação entre processos

- Sincronização: send e receive podem ser bloqueantes (síncronos) ou não (assíncronos)
 - Envio bloqueante: emissor é bloqueado até a mensagem ser recebida pelo processo ou caixa postal
 - Não bloqueante: emissor envia e continua execução, sem aguardar recebimento
 - Recebimento bloqueante: receptor fica bloqueado até que exista mensagem disponível para recebimento
 - Não bloqueante: receptor recebe mensagem válida ou nula

Comunicação entre processos

- Bufferização: na comunicação direta ou indireta, mensagens residem em uma fila temporária (buffer)
 - Buffer pode ter:
 - Capacidade zero: o link não pode manter mensagens esperando
 - Emissor deve ficar bloqueado até receptor obter mensagem
 - Capacidade limitada: tamanho finito n
 - Enquanto fila não está cheia, emissor pode enviar e continuar execução sem ficar bloqueado
 - Quando fila está cheia, emissor deve ficar bloqueado
 - Capacidade ilimitada
 - O emissor nunca fica bloqueado

Comunicação em sistemas cliente-servidor

- Técnicas para comunicação em sistemas cliente-servidor:
 - Memória compartilhada
 - Passagem de mensagens
 - Sockets
 - RPC
 - Pipes

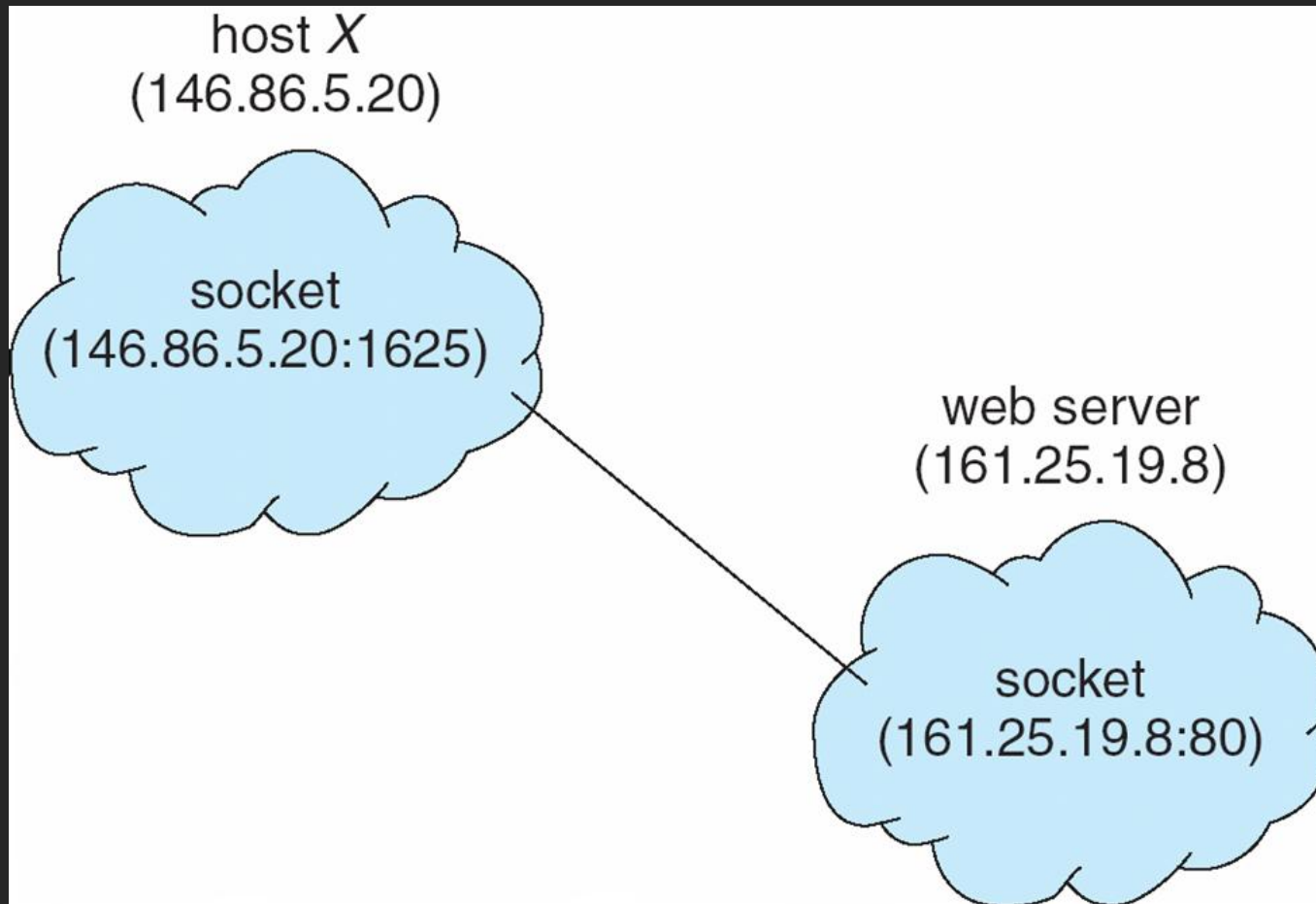
Comunicação em sistemas cliente-servidor

- Um socket é definido como uma extremidade para comunicação
 - Endereço IP + porta de comunicação
 - Identifica unicamente uma aplicação (porta) em uma máquina (endereço IP)
 - O soquete 161.25.19.8:1625 refere-se à porta 1625 no host 161.25.19.8.
 - Algumas portas são bem conhecidas (<1024): 20 e 21 (servidor ftp), 22 (ssh), 23 (telnet), 25(e-mail), 80 (http), 443 (https), 53 (dns), etc.

Comunicação em sistemas cliente-servidor

- (cont.)
 - A comunicação entre máquinas A e B necessita de um par de sockets (socket pair)
 - Um socket em cada máquina
 - O par de sockets identifica unicamente uma comunicação
 - Operações: criação, término, leitura e escrita socket
 - Tipos de sockets: orientados a conexão (TCP), sem conexão (UDP), multicast

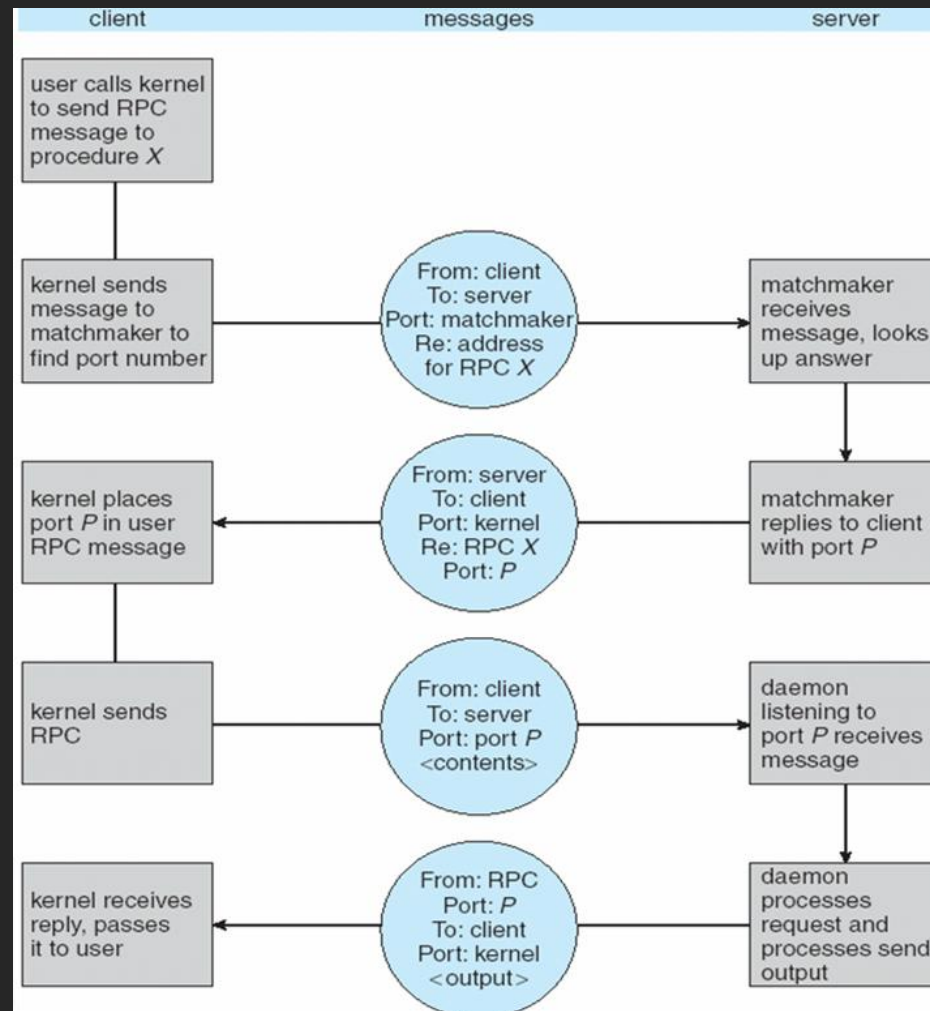
Comunicação em sistemas cliente-servidor



Comunicação em sistemas cliente-servidor

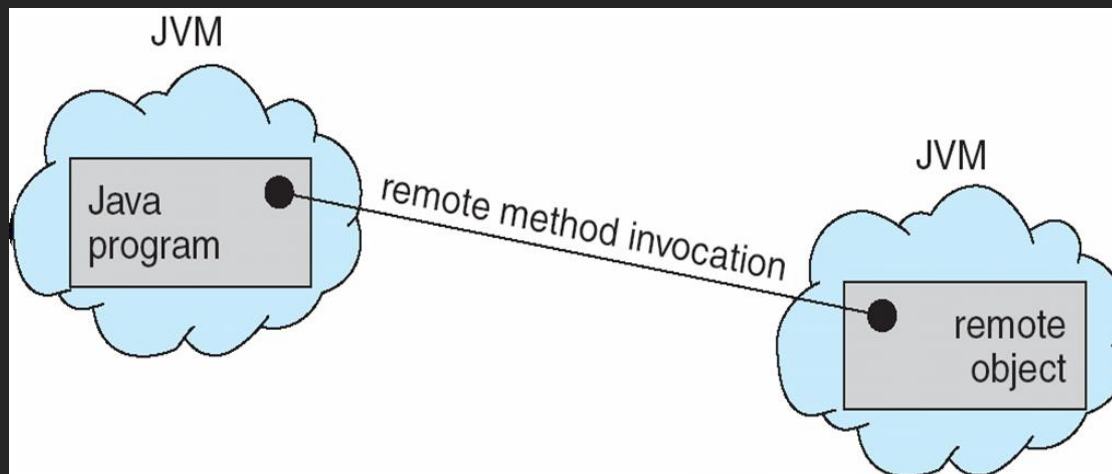
- Chamada remota de procedimento (RPC): abstrai chamadas de procedimentos entre processos em um sistema de rede
 - Stub cliente: proxy/substituto do lado cliente para o procedimento no servidor
 - Localiza o servidor e envia parâmetros empacotados
 - Stub servidor: recebe a mensagem, desempacota os parâmetros e executa o procedimento no servidor
 - Serviço RPC pode ser disponibilizado em portas estáticas ou dinâmicas
 - Dinâmicas: necessidade do matchmaker em porta fixa

Comunicação em sistemas cliente-servidor



Comunicação em sistemas cliente-servidor

- Invocação remota de método (RMI): mecanismo Java similar a RPCs
 - Permite um programa em Java invocar métodos em objetos remotos



Comunicação em sistemas cliente-servidor

- Pipe: canal de comunicação entre dois processos
 - Um dos primeiros mecanismos de IPC do UNIX
 - Forma simples (e limitada) de comunicação entre dois processos
 - Tipos:
 - Pipes ordinários: comunicação na forma produtor-consumidor
 - Unidirecional: um processo escreve e outro lê
 - Usado para processo pai se comunicar com filho
 - Existe somente enquanto processos existem
 - Pipe nomeado (FIFO): mais elaborado
 - Bidirecional, não necessita de relação pai-filho, existem até que sejam apagados do sistema de arquivos

Comunicação em sistemas cliente-servidor

- (cont.)

- Ex.:

- Linux: ls | more
 - Programa ls é produtor e more é consumidor
 - Windows: dir | more